Anchor Evaluation — 평가코드

어텐션이 원인 로그를 찾는가? — 랭킹 지표로 GO / NO-GO 판정

이 코드가 검증하는 것

가설 : 에러 줄의 어텐션이 ' 진짜 원인 줄 ' 을 가리킨다

- 에러 (ERROR/WARN) 줄을 기준 (anchor) 으로 , 그 어텐션 점수로 이전 줄들을 줄 세운다
- 진짜 원인 줄이 위쪽에 오는지 채점한다

중요 : 압축 성능을 재는 게 아니다

- 압축기를 만들기 전에 , 기댈 ' 어텐션 신호 ' 가 유효한지부터 검증하는 단계
- 통과 (GO) → 그 신호로 압축기를 만든다 / 실패 (NO-GO) → 가설 수정

파이프라인

① 점수
`score_fn`

후보에 점수 매김
(어텐션 / 거리 / 무작위)

② 정렬 → 0/1
`ranked_labels`

점수로 줄 세워
정답여부 리스트 [1,1,0,..]

③ 지표
`recall·AP·AUC..`

0/1 리스트에서
1 이 얼마나 위에 몰렸나

④ 집계
`MAP`

앵커 → 장애 →
전체 2 단 평균

⑤ 통계
`bootstrap CI`

우리− baseline 차이
CI 가 0 넘나

⑥ 판정
`decide`

GO /
CONDITIONAL /
NO-GO

핵심 : 모든 지표는 결국 ' 정렬된 0/1 리스트 ' 를 세는 것뿐 — 어려운 수학이 아니라 ' 줄 세우고 세기 '

우리 (Anchor) + baseline 4 개

방법	무엇을 하나	무슨 딴지를 막나
Anchor (우리)	ERROR/WARN 줄의 어텐션으로 이전 줄 랭킹	증명하려는 주인공
Position-matched INFO	평범한 INFO 줄을 기준으로 바꿔서 비교	ERROR/WARN 이라 잘 찾나 ? 위치 때문 ?
Recency	어텐션 무시 , 그냥 가까운 줄 우선	그냥 가까워서 맞는 거 아냐 ?
All-Query	여러 줄의 query 어텐션을 평균	한 줄 집중이 의미 있나 ?
Random	무작위	우연 수준 바닥선

ranked_labels — 점수 → 정렬 → 0/1

```
def ranked_labels(candidates, anchor, score_fn):
    scored_pairs = []
    for c in candidates:
        score = score_fn(c, anchor)   # 점수 매김
        scored_pairs.append((score, c['is_groundT']))

    scored_pairs.sort(reverse=True)   # 점수 높은순 정렬

    labels = []
    for pair in scored_pairs:
        labels.append(1 if pair[1] else 0)
    return labels
```

결과 예 : [1, 1, 0, 0, 0] → ' 어텐션으로 줄 세웠더니 1·2 등이 정답 '

하는 일

① 후보마다 (점수 , 정답여부) 짝 만들기

② 점수 높은 순으로 정렬

③ 정답여부만 뽑아 0/1 리스트로

score_fn 만 바꿔 끼우면

우리 / recency / random 을
한 코드로 다 돌림

0/1 리스트를 세는 5 가지 방법

Recall@K	전체 정답 중 top-K 에 든 비율	정답을 얼마나 '건졌나'
Precision@K	top-K 중 진짜 정답 비율	고른 것이 얼마나 '정확한가'
MRR	첫 정답이 몇 등 ? 의 역수	정답이 얼마나 위에 있나
AP	정답마다 precision 재서 평균	정답을 위로 몰수록 높음
AUC	정답이 오답보다 위인 비율	전체 순서가 얼마나 옳은가

MAP — 2 단 평균

데이터가 3 층으로 겹쳐 있다

```
0/1 리스트        = 앵커 1 개 결과        [1,1,0]
       ↓ 여러 개 모이면
anchor_label_lists = 장애 (incident) 1 개   [[1,1,0],[0,1,0]]
       ↓ 또 여러 개 모이면
incidents          = 전체 장애들
```

incident_ap: 장애 안에서 앵커들 AP 평균 → 장애 점수 1 개

MAP: 장애들 점수를 다시 평균 → 전체 종합 점수 (주 지표)

왜 2 단 ? 앵커 많은 장애가 결과를 독차지 못 하게 — 장애당 점수 1 개로 공평하게

차이가 진짜냐 — 부트스트랩 CI

delta_map = 우리 MAP − baseline MAP (차이)

bootstrap_ci: 장애를 복원추출로 다시 뽑아 차이를 5,000 번 계산 → '얼마나 흔들리나' 범위 (CI)

오차율을 사람이 정하지 않는다. 데이터의 흔들림이 '같음'의 경계를 정한다.

CI 위치로 판정

CI 전체 > 0 → 이김

CI 전체 < 0 → 짐

CI가 0을 포함 → 같음

차이값 = 우리 − baseline 이라, 0 = 차이 없음 = 같음

decide — GO / NO-GO

```
def decide(dmap_allq, p_allq,
           dmap_recency, p_recency):
  both_positive = dmap_allq>0 and dmap_recency>0
  both_sig = p_allq<0.05 and p_recency<0.05
  if both_positive and both_sig:
      return 'GO'
  if  한쪽만 통과 :
      return 'CONDITIONAL'
  return 'NO-GO'
```

GO

두 비교 모두 우위 + 유의

CONDITIONAL

한쪽만 통과

NO-GO

둘 다 미달

※ 비교 많으면 BH-FDR 보정

실제 실행 결과

우리 (어텐션): [1, 1, 0, 0, 0]

`Recall@3 = 1.0  Precision@3 = 0.67  MRR = 1.0  AP = 1.0  AUC = 1.0`

→ 정답 둘을 1·2 등에 올림 (완벽)

recency: [0, 1, 0, 1, 0] AP = 0.5

→ 정답이 흩어짐 (가까운 줄 기준으론 못 찾음)

→ 어텐션 (AP 1.0) 이 recency(AP 0.5) 보다 원인을 위로 올렸다 = 신호가 유효하다는 증거

실제 Nezha 데이터 + baseline 4 종으로 돌려서 GO/NO-GO 를 내는 것이 다음 단계

week7(측정) → week5(판정) 연결

1 층 보존율 평가 — 로직은 완성 , 입력만 연결하면 끝

```
week7 ( 측정 )                          week5 ( 판정 )
압축기 출력 파싱 → 생존 배열   ———→   cell 0 입력 교체
(KEEP/DROP → 원인 줄 0/1)               preservation·McNemar·BH ( 이미 완성 )
```

할 일

- week5 cell 0 의 하드코딩 배열 ([1,1,0,1]) → week7 이 뽑는 실제 생존 배열로 교체
  - 원인 줄 정답 (cause·is_error·pos) 은 데이터셋 라벨로 고정 (측정값으로 정하지 않기 = 누수 방지)
  - 압축률 (50/60/70/80/90%) 마다 반복 + baseline 4 종 채우기
- LogPress 자체 생존 배열은 C 압축기 완성 대기

2층 평가 (효율 / 비용) — 보충 설계

GPU 피크 · 베이스라인 분류 · forward 비용 처리 · 파레토 '같음' 판정

1. 개념

GPU 피크란?

- **추론 중 GPU 메모리를 가장 많이 쓴 그 순간의 사용량** (peak = 최고점)
- 코드로는 `torch.cuda.max_memory_allocated()` — 돌리는 동안 할당된 메모리의 **최댓값**
- GPU는 그 순간 못 버티면 터짐(OOM) → "이 GPU에 올릴 수 있냐"는 **평균이 아니라 피크**로 결정됨
- 우리 압축기가 KV를 줄이면 이 피크가 내려감 → 같은 GPU에 더 긴 로그 / 더 많은 요청 수용

> ⚠️ 피크 실측값엔 KV만이 아니라 **모델 가중치·활성화까지 섞여서** 잡힌다. → 순수 KV 절감은 **이론 계산값**으로 보고, 피크 실측은 "실제로도 안 터지고 줄더라" 확인용으로 사용.

2. 베이스라인

무엇으로 KV를 남기나

베이스라인	KV 선택 기준	어텐션 점수	동적/정적	eager forward
SnapKV	최근 쿼리의 어텐션 점수 → 중요한 prefix 토큰 top-k	✅ 읽음	동적	필요
H2O	누적 어텐션 점수 높은 토큰(heavy hitter) + 최근	✅ 읽음	동적	필요
ObservedAttention	관측된 어텐션 가중치로 점수화 후 제거	✅ 읽음	동적	필요
StreamingLLM	맨 앞 sink 토큰 + 최근 윈도우 (**위치 고정**)	❌ 안 읽음	정적	불필요
random KV	무작위 제거	❌	정적(무작위)	불필요
level filter (우리 비교군)	로그 레벨(ERROR/WARN 등)	❌	정적	불필요
우리 압축기	로그 신호(레벨·희귀도·위치) + 어텐션	✅ 읽음	동적	필요

2. 베이스라인

두 부류로 갈림

어텐션 점수 기반 (동적)

SnapKV · H2O · ObservedAttention · 우리

런타임에 어텐션 점수를 읽어 남길 KV를 고름
→ 점수를 메모리에 띄우는 **eager 어텐션 필요**

위치·규칙 기반 (정적)

StreamingLLM · random KV · level filter

고정 규칙으로 남김 (위치·무작위·레벨)
→ 어텐션 점수 안 읽음 → **eager 어텐션 불필요**

용어 주의: "어텐션 기반"은 두 뜻으로 섞임 — 넓게(어텐션 위에 세움) vs 좁게(런타임에 점수 읽음). forward 비용에선 **좁은 뜻(점수 읽음)**이 중요 → "점수 기반 / 위치·규칙 기반"으로 표기.

비교 상대별 방법

비교 상대	방법
점수 기반 SnapKV · H2O · ObservedAttention	forward가 양쪽 공통 → **소거 지름길 OK** (또는 끝까지 실측해도 자동 상쇄)
위치·규칙 기반 StreamingLLM · random KV · level filter	소거 지름길 쓰지 말 것. 끝까지 **실측값 그대로** 사용 → 우리 eager forward 오버헤드가 그 숫자에 정직하게 잡히게 둠

원칙

"끝까지 실측하면 공통분은 저절로 상쇄된다."
점수 기반끼리만 'forward 빼자' 지름길을 쓰고, 위치·규칙 기반엔 실측값을 그대로 둔다 (빼주면 우리에게 유리하게 조작한 셈).

4. § 5 합격 기준

파레토 '같음' 판정 (오차율 처리)

오차율은 숫자로 정하지 않는다. "5% 이내면 같음" 같은 임의 기준 대신, 여러 번 재서 흔들림(CI)이 **0을 걸치는지**로 판정.

- ① **측정** — 차이값(우리 − baseline)을 표본으로 모은다. Nezha 장애 케이스별로 LogPress/baseline 보존율 차이 → baseline마다 38개 케이스 = CI 1개
- ② **계산** — 그 표본으로 **부트스트랩 95% 신뢰구간(CI)**을 뽑는다. CI = 측정값들로 계산한 "범위"
- ③ **판정** — CI가 0을 기준으로 어디 있는지 본다 ↓

CI 위치	판정
CI 전체 > 0	이김
CI 전체 < 0	짐
CI가 0을 포함	**같음**

파레토 합격

두 축(보존율·메모리) 모두 '짐'이 없고, 최소 한 축에서 '이김'.

4. § 5 합격 기준

표본이 뭔지 + 정할 숫자

- **보존율 축:** 표본 = **케이스들** (케이스마다 보존율 차이 → 그 묶음으로 CI)
- **메모리·속도 축:** 표본 = **반복 측정 횟수** (같은 조건 N번 재서 그 묶음으로 CI)
- ※ 베이스라인 5개는 표본이 아니라 **5번의 1:1 비교** → 각자 CI. 비교가 많으므로 '이김' 선언엔 **BH-FDR 보정** (1층과 동일).

숫자	뭐냐	값
케이스 수 (Nezha 정제 필요)	보존율 축 표본	20~30+ (많을수록 CI 좁아짐. 2개면 다 '같음')
측정 반복 N	메모리·속도 축 표본	속도 10회↑ / 메모리 3회 (워밍업 제외)
부트스트랩 B	CI 계산 내부 재표집	5,000 (코드 상수, 1층과 동일)

핵심: 오차율을 사람이 정하지 않고 **데이터의 흔들림이 '같음'의 경계를 정한다.** 값들은 관례 수준 — 중요한 건 **실험 전에 정해서 프리즈** (결과 보고 바꾸면 어뷰징).

5. 측정

CI 뽑는 순서 — 재표집

0. 입력 — 각 케이스에서 우리 보존율 − baseline 보존율 → 길이 38 배열

```
diff = [+0.05, 0.00, −0.03, +0.02, ...]   # 길이 38
```

재표집 = "같은 칸을 복원추출로 다시 채움" (중복 OK, 누락 OK) — 설명용 6칸

```
resample #1 = [+0.05, +0.05, 0.00, +0.02, −0.01, +0.04]   mean = +0.0250
resample #2 = [−0.03, 0.00, +0.02, +0.02, +0.04, +0.05]   mean = +0.0167
resample #3 = [+0.04, +0.05, +0.02, 0.00, −0.03, +0.02]   mean = +0.0167
  ... 5,000번 반복 → 평균 5,000개
```

원본 38칸을 복원추출로 다시 채워 **평균 1개** → 5,000번 = 평균 5,000개. 이 5,000개를 정렬해 **2.5% · 97.5% 자르면 95% CI.** (새 데이터를 더 재는 게 아니라, 있는 걸 섞어 "평균이 얼마나 흔들리나"를 보는 것)

5. 측정

"CI가 0을 기준으로"의 뜻

- CI 안의 값은 전부 **차이값 = (우리 − baseline)** → 그래서 **0 = 차이 없음 = 같음**이 기준점
- 값이 **양수** = 우리가 더 높다 / **0** = 같다 / **음수** = 우리가 더 낮다

```
[+1.2, +3.5]   범위가 통째로 0보다 큼    → 어떤 값이든 우리가 높음  → 이김
[−3.5, −1.2]   범위가 통째로 0보다 작음  → 어떤 값이든 우리가 낮음  → 짐
[−1.0, +2.0]   범위가 0을 걸침           → +일 수도 0일 수도 −일 수도 → 같음
```

마지막이 '같음'인 이유: 진짜 차이가 +2일 수도, 0일 수도, −1일 수도 있어서 **"우리가 확실히 다르다"고 말할 수 없음** → 사실상 같다고 본다.

요약 — 차이값이 "우리 − baseline"이라 **0이 '같음' 지점.** CI(차이의 범위)가 0 위 / 0 아래 / 0 걸침 → 이김 / 짐 / 같음.

주간 요약

LogPress 진행 현황

11월 2일 발표자료의 Part A 결과에, 11월 9일까지의 Part B 분석 내용을 추가했습니다.

Part A

Baseline 4종 공통 평가 연결 완료

LineScore · 36 Layer 추적

Thunderbird 65줄 샘플 평가

Part B

Nezha Exception 3건 분석 완료

증거 로그 후보 116행 통합

CSV 중복 1행 확인

다음 작업

중복 제거 · 공통 파서 정리

나머지 장애 유형 자동 탐색

후보 검증 및 입력 라인 매핑

이번 보고의 핵심

- Part A: 4개 Baseline의 통합 평가 결과는 지난 자료와 동일하게 유지했습니다.
- Part B: 공식 장애 시각·Pod와 CSV 로그를 연결하고, TraceID로 파급 오류를 추적했습니다.
- 주의: 116행은 공식 RCA GT가 아닌 미검증 후보이며, 중복 제거도 아직 반영 전입니다.

Part A 진행 상황

완료한 내용

- ObservedAttention · SnapKV · StreamingLLM · H2O를 공통 Evaluator에 연결했습니다.
- Qwen2.5-3B 모델의 36개 Layer와 KV Head별 생존 토큰을 추적했습니다.
- StreamingLLM의 Sink/Recent, H2O의 Heavy Hitter/Recent 동작을 검증했습니다.

공통 평가 흐름

원본 토큰 위치
KV 생존 토큰

→

로그 라인 점수
LineScore

→

Top-K 선택
라인 예산

→

정답 비교
GT 보존율

공통 코드

- line_mapping.py : 토큰 ↔ 로그 라인 연결
- line_score.py : Layer × KV Head 생존율 평균
- evaluator.py : Top-K 및 정답 보존율 계산

이번 실험 조건

- Qwen2.5-3B-Instruct · 36 Layers · KV Heads 2
- Thunderbird 로그 65줄 · 이상 GT 17줄
- KV 압축률 0.5 / 0.7 / 0.9

Baseline 평가 결과

Thunderbird 단일 샘플 · Anomaly Line Retention

동일한 로그 65줄 / 이상 로그 17줄 / LineScore 기반 Top-K

Baseline	압축률 0.5	압축률 0.7	압축률 0.9
StreamingLLM	**8 / 17 (47.06%)**	**5 / 17 (29.41%)**	**0 / 17 (0%)**
H2O (Prefill 변형)	7 / 17 (41.18%)	4 / 17 (23.53%)	1 / 17 (5.88%)
ObservedAttention	6 / 17 (35.29%)	4 / 17 (23.53%)	0 / 17 (0%)
SnapKV	6 / 17 (35.29%)	2 / 17 (11.76%)	1 / 17 (5.88%)

이번 결과에서 확인한 점

- StreamingLLM이 압축률 0.5와 0.7에서 가장 높았지만, 한 샘플의 결과입니다.
- 이번 정답 17줄은 Thunderbird의 이상 로그이며 RCA 원인·증거 GT는 아닙니다.
- H2O는 원본의 순차적 eviction이 아닌 prefill attention 기반 변형입니다.

평가상 주의 Top-K로 선택한 라인은 실제 KV에 완전한 문장이 남았다는 뜻은 아닙니다. KV 압축률과 line budget 대응도 최종 검토가 필요합니다.

우선순위 판단 | 왜 Part B를 먼저 진행했나?

Part C 지원을 포기한 것이 아니라, 최종 RCA 평가에 필요한 데이터 기반을 먼저 마련한 선택입니다.

Part A에서 이미 확보한 것

Baseline 4종을 같은 방식으로 실행·비교하는 틀

- 단일 Thunderbird 샘플에서 이상 로그 보존율 확인
- 다만 이상(anomaly) 라벨은 RCA 증거 GT가 아님

최종 평가에서 아직 부족했던 것

Nezha 장애별 원인·증거 로그 줄의 검증된 라벨

- fault_list.json은 주입 시간·Pod·유형만 알려줌
- 정답 줄이 없으면 실제 RCA 보존율을 산정하기 어려움

프로젝트 전체의 연결 순서

Part A
공통 Baseline 평가 틀

→

Part B
장애·로그 매핑과 GT 검증

→

Part C
로그 인지형 압축 설계

→

Part D
동일 기준의 성능 평가

이번 주 우선순위의 이유

Part B의 검증 가능한 라벨·라인 매핑을 먼저 만들면, Part C 개선 효과를 추측이 아닌 보존율로 비교할 수 있습니다.

Part C의 독립 개발은 병행 가능하며, Part B의 데이터 규격이 안정되면 평가 연동과 기술 지원을 진행합니다.

Part B | Nezha 데이터셋 정제 과정

공식 장애 정보와 실제 로그를 연결해 RCA 증거 로그 후보를 생성했습니다.

처리 흐름

01
공식 장애 정보
fault_list.json
시간 · Pod · 장애 유형

→

02
CSV 로그 추출
log/*.csv
장애 전후 60초

→

03
오류·요청 연결
Severity / TraceID
직접·파급 오류 탐색

→

04
후보 JSON 저장
Incident별 파일
로그 행 위치 기록

확인한 원본 구조

- rca_data/2022-08-22/의 log · metric · trace 구성 확인
- fault_list.json: 장애 주입 시각, 대상 Pod, 유형 제공
- 실제 로그: Timestamp · PodName · TraceID · Log 컬럼
- 서비스별 로그 형식이 달라 JSON 및 텍스트 파싱 필요

분석하면서 확인한 점

- CSV 이름의 시각과 실제 로그 시각이 정확히 일치하지 않음
- adservice는 INFO만 남아도 frontend에 오류가 나타남
- 원본 CSV 행 번호는 모델 입력의 로그 라인 번호와 다름
- fault_list에 공식 원인·증거 로그 라벨은 없음

Part B | Exception 장애 3건 분석 결과

2022-08-22 Nezha · 장애 발생 후 60초 내 ERROR 후보를 탐색하고 요청 단위로 연결

증거 로그 후보 추출 결과

주입 대상	오류 기록 서비스	후보 로그	TraceID	대표 메시지
frontend	frontend	**42행**	21개	주문 처리 실패 (2종)
checkoutservice	checkoutservice	**27행**	27개	Failed to connect payment service
adservice	frontend	**47행**	46개	Failed to retrieve ads

합계 116행 · Incident별 TraceID 총 94개 (중복 제거 전)

adservice 사례: 장애가 다른 서비스에 드러남

adservice
ERROR 0 / INFO 1,353

→

frontend
광고 조회 ERROR 47개

47개 모두 adservice 로그와 TraceID·선후 관계 일치

중복·GT 주의사항

서로 다른 두 CSV에 동일한 adservice ERROR 1건이 중복 저장됨.

116행 → 확인된 중복 1행 제외 시 115행. 아직 JSON 미반영.

모두 candidate_unverified

Part B | 주요 파일과 다음 작업

이번 주 생성한 코드는 탐색용 분석 스크립트와 결과 저장·통합 스크립트로 구성됩니다.

주요 Python 파일 역할

① 구조 파악

inspect_faults.py	장애 주입 목록·유형 확인
inspect_logs.py	CSV 컬럼과 실제 시간 범위 확인
inspect_incident.py	첫 CPU 장애의 로그 확인

② 장애·TraceID 분석

inspect_exception.py	frontend 오류 분석
inspect_checkout_exception.py	checkoutservice 오류 분석
inspect_ad_exception.py	adservice 파급 오류 분석·저장
inspect_trace.py	같은 TraceID의 서비스 흐름 추적
inspect_error_detail.py	오류 메시지의 내부 필드 확인

③ 후보 JSON 생성 및 통합

build_evidence_candidates.py	frontend 후보 JSON 생성
build_checkout_candidates.py	checkoutservice 후보 JSON 생성
build_dataset.py	3건 병합 · 중복 진단

현재 산출물

Part_B/week12/output/

frontend_exception_candidates.json

checkout_exception_candidates.json

adservice_exception_candidates.json

nezha_exception_dataset.json

다음 우선순위

1. 중복 제거 및 공통 파서 구현
2. 전체 장애 유형으로 후보 탐색 확장
3. GT 검증 · 입력 line_index 연결

남은 작업 | Part A · Part B 전체 로드맵

완료: Part A 1차 Baseline 연결 / Part B Exception 후보 생성. 미완료: RCA 기준의 신뢰 가능한 평가.

Part A | Baseline 평가의 타당성·확장

1. **보존율 해석 정리**
   LineScore Top-K와 실제 KV에 남는 정보 구분
2. **공정한 압축 예산**
   KV 토큰 압축률과 로그 라인 선택 예산 비교 기준 확정
3. **Baseline 재현성 검토**
   H2O prefill 변형의 한계 명시·설정과 실행 기록
4. **평가 확대**
   Nezha 다중 장애·모델 확장·메모리/시간 기록

Part B | GT 구축과 평가 데이터 전달

1. **원본 정합성·중복 처리**
   CSV 겹침 검사, 중복 제거, JSON/텍스트 공통 파서
2. **장애 유형 확장**
   2022-08-22의 24건 탐색: CPU·네트워크는 Metric/Trace 확인
3. **증거 후보 검증**
   직접 오류/파급 오류 구분, 수동 검수 후 GT 확정
4. **모델 입력 연결·분리**
   CSV 행 → 입력 line_index, Incident별 분리·loader 작성

팀 간 연결 지점

B: 검증 GT·라인 매핑·loader 제공 → A: 동일 데이터로 Baseline 재평가 → C: LogPress 개선 → D: 최종 RCA 성능 비교